科目：データベースの基礎

第6回 リレーショナルデータベース設計

講師：鈴木源吾

小テスト1の返却

- 解答用紙を返却する．解答例は WebClass「小テスト1 結果・解答例」に PDF で掲載
  - 講義ルールの「小テスト正解の撮影禁止」は**削除**（解答例を配るため）．他の人への共有・再配布の禁止は変わらない
- 得点は部分点（0.5点単位）を合計して**切り捨て**．得点欄の右のかっこ内が切り捨て前の点
- **7点未満（欠席を含む）の人**：WebClass「小テスト1 レポート」を提出する
  - 内容：小テスト1と同じ問題を，ポイント資料を見ながら解き直す（レポート用紙は WebClass に添付）
  - 締切：**11/11 講義開始30分後**（満点7点）．締切後は 2/19 18:30 まで提出でき，満点6点
- 小テスト2は **11/25**．範囲は第4〜7回．持ち込みは第4〜7回のポイント資料のみ

前回の復習

1. SELECT文：前回の続き
   - AND・OR，IS NULL，LIKE．複数テーブルの結合（JOIN … ON，ふつうは主キー＝外部キー）
   - ORDER BY，集約関数（COUNT・SUM など），GROUP BY
2. SQLとリレーショナル代数の対応
   - SELECT 句が射影 π，WHERE 句が選択 σ，JOIN … ON が結合 $\bowtie$．SQL の結果は重複を許す
3. （参考）テーブルとデータの作成・削除・更新：CREATE TABLE，INSERT，UPDATE，DELETE

第5回演習の解説

- 第5回ポイント資料の裏面（演習1・演習2・確認問題）の正解を解説する
- 解説は，演習の締切（講義開始30分後）のあとに行う
- 解答例は電子的には配布しない．**自分の解答に転記すること**（解説中の撮影は不可）
  - 講義中に自分の解答へ書き足す・訂正するのはよい．そのまま試験に持ち込める
- 提出がまだの人は，WebClass の「【締切後】第5回 演習」から提出する（0.7点で評価）

お知らせ：提出ファイル名は自由に

- これまでの「ファイル名の先頭は学籍番号」は**やめる**
- 撮影したファイルを，**名前を変えずにそのまま** WebClass に提出してよい
  - 提出者は WebClass が学籍番号・氏名で管理するので，採点に支障はない
- 引き続き守ること：**表面は問1，裏面は問2**（両面をまとめた PDF は問1 に提出してよい）
- 演習だけでなく，小テストのレポートも同じ

今回のトピック

1. データベースの設計とは？（設計プロセス：概念設計・論理設計・物理設計）
2. 実体関連モデルの概要（実体・関連・属性・主キー）
3. 実体関連モデルの詳細（多重度・汎化階層・表記法の違い）
4. 今回の演習と小テストの「型」：**表から ER 図を描く**

教科書：第6章 p.63〜72（6.1 実世界の概念モデリング p.63，6.2 実体―関連モデル p.65，6.3 弱実体型と実体―関連モデルの拡張 p.68）

Topic 1

データベースの設計とは？

トピックのキーワード

- スキーマを決めること
- 要件定義
- 概念設計・論理設計・物理設計

データベースの設計

「データベースの構造と制約，つまり，スキーマを決めること」

- リレーショナルデータベースなら
  - テーブルを決める
  - 属性を決める
  - 型を決める
  - 主キーを決める
  - 外部キーを決める
  - ・・・
- リレーションスキーマは，これまでリレーションの構造という意味で使用したが，一般にスキーマは構造だけではなく，制約も含めて考えることが多い

なぜデータベースの設計が必要？(1/3)

- 「設計」しなくても，ただ属性にするデータ項目を集めただけでもテーブルを作ることはできる

しかし，設計なしでは，以下のような問題が生じることがある

- 他の人がそのテーブルをみたときに，どんな情報であるかわからない
- 似た名前の属性があるが，それぞれが何を意味するかわかりにくい
- 違う名前の属性だが，実は同じ意味を表しており，重複している
- 検索したい属性がどこにあるのかわかりづらい
- 制約が適切に設定されておらず，データに矛盾が生じてしまう

→　適切なデータベースにするためには，きちんとした設計が必須！

なぜデータベースの設計が必要？(2/3)

- データベースの設計を「**意識する**」ことが重要
- 例）データベース設計あるある
  - XさんがA業務のためにデータベースを構築
  - B業務にもそのデータベースを使いたいと要望がでる
  - B業務に使うには，構造の見直しが必要
  - A業務のアプリケーションの★大規模修正★
    - 構造変更の反映は大作業

なぜデータベースの設計が必要？(3/3)

- 何が悪かったのか，どうすればよかったのか？
- データベースは「**複数の応用**」に使うことを目的としている（第1回：データベースの定義）
- だから，**特定の業務やアプリケーションと独立**にデータベースを設計する必要がある
  - これは理想論ではある．実際には完全に独立は難しい
- **みんなが使えて**，（できれば）ずっと**変更なし**で使える設計が必要

データベースの設計の進め方

データベースの設計のために，決めなければならないことはとても多い

- 業務を知る必要がある．業務は複雑でルールも多い
- 業務に詳しい人が，データベースに詳しいとは限らない

→ データベースの設計の前に，業務を整理する必要がある（要件定義）

→ データベースの設計を段階的に行う必要がある（設計プロセス）

- 「抽象」→「具体」へ
- 「業務」→「処理」へ
- 「業務に詳しい人」→「データベースに詳しい人」へ

要件定義

「要件定義」とは，アプリケーション開発全体の中で（データベースを設計する前段として），以下を行う工程である

- 利用者からの要求事項の抽出や整理
- 業務が対象とする情報の洗い出し

→　**ただし，要件定義は次に述べる概念設計との関連も強くその境界は曖昧**

データベースの設計プロセス

データベースの設計は，以下の3段階のステップで行っていく

順	段階	作るもの（例）
1	**概念設計**	概念モデル（概念スキーマ）：ER 図
2	**論理設計**	論理モデル（論理スキーマ）：テーブル・属性・主キー・外部キー
3	**物理設計**	内部スキーマ（物理スキーマ）：インデックス，格納場所，ファイルの形式

- 抽象（実世界）→ 具体（DBMS 上の格納）の順に進む

データベースの概念設計

- 実世界を概念的・理想的に記述する概念モデル（概念スキーマ）を作成する
- 実際のデータベースで使用するデータモデルやスキーマとは独立に作成する
- 実装にとらわれると，理想的な整理ができないと考えられるからこの工程がある
- 利用者の要件や情報システムの発展内容から情報構造を総合的に記述
- **今回学ぶ ER 図（実体関連図）は，概念設計で使う**

データベースの論理設計

- 論理モデル（論理スキーマ）を作成する
- 実際に使用するDBMSにあわせた，論理的な構造を記述する
  - リレーショナルデータベースなら，テーブル・属性・型・主キー・外部キーを決める
- どのように格納するかという物理的な構造ではない
- 次回学ぶ**正規化**は，論理設計で「良い表」にするための指針

データベースの物理設計

- 内部スキーマ（物理スキーマ）を作成する
- 実際に物理的に格納したり管理するための構造を記述する
- 性能面や運用面から，必要な情報を分割したり，インデックスを付加したり，情報を冗長化したりすることを行う
- 格納する磁気ディスクの場所，データを格納する領域，ファイルの形式，各種パラメータの設定，等

Topic 2

実体関連モデルの概要

トピックのキーワード

- 実体（実体型）・関連（関連型）
- 属性・主キー
- ER 図（Chen の記法）

概念設計と実体関連モデル

実体関連モデル（ERモデル）は，基本的に実世界は，「実体（entity）」と実体間の「関連（relationship）」で表せるという考え方に基づいている



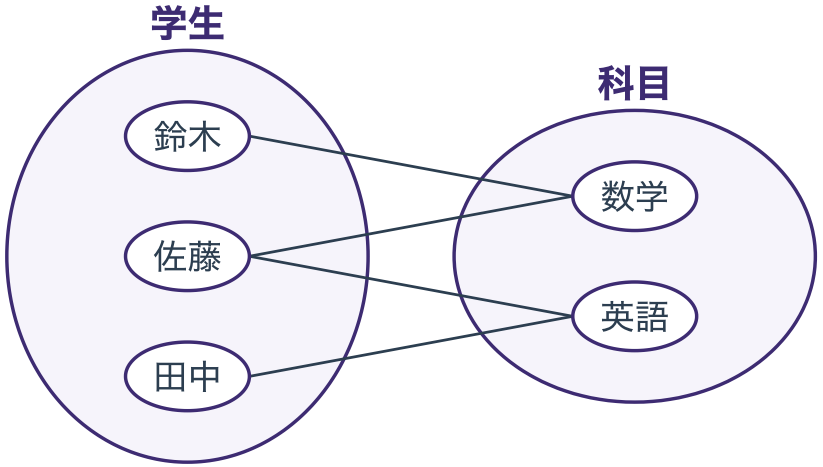
実体型（四角）　　関連型（ひし形）

- 実体の集まりを四角で表す → **実体型**（または実体集合）と呼ぶ
- 関連の集まりをひし形で表す → **関連型**（または関連集合）と呼ぶ
- 実体関連モデルで表現した結果を **ER図**（実体関連図）と呼ぶ．情報の構造をビジュアライズでき，実世界をモデル化する概念設計に向いている

実体と関連



ER 図



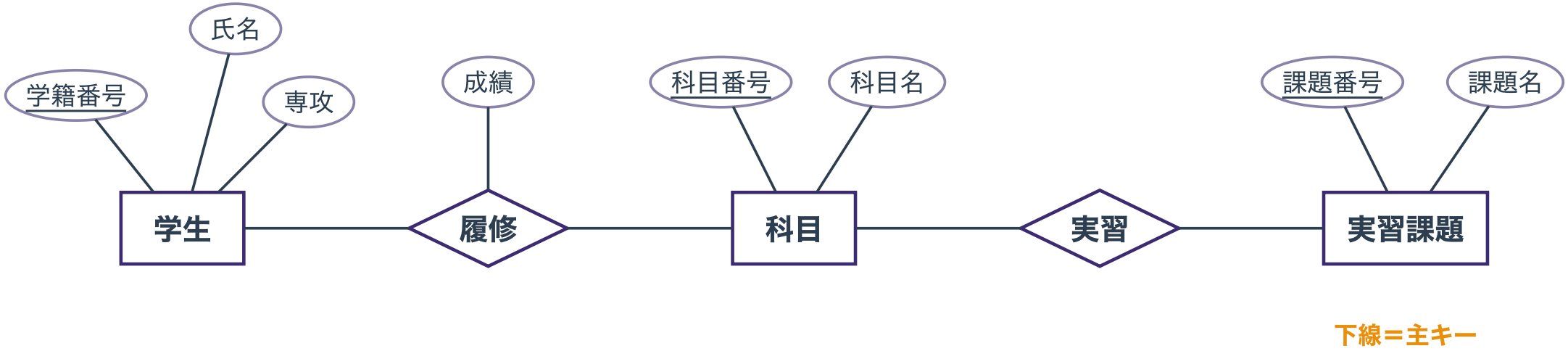
対応する現実世界のイメージ

- ER図が表すこと　→　学生は，科目を履修している
  - 以下の現実世界を「抽象的に」捉えている
- 対応する現実世界のイメージ
  - 鈴木は，数学を履修している．佐藤は，数学と英語を履修している．田中は，英語を履修している

実体関連モデルの構成要素（1/2）

- 実体 (Entity)
  - データを管理する「対象」
  - 学生，会社，顧客，商品，売上，等々
- 関連 (Relationship)
  - 実体の間の論理的，意味的な関係
  - 学生は大学に「属する」
  - 会社は商品を「販売する」，等々
- ある概念を「実体」にするか，「関連」にするか，どちらも選べることがある
  - 例）前ページの例の「履修」は関連としたが，実体にすることもある
  - 関連が，属性（次ページで説明）を多く持つときに，実体にすることがある

実体関連モデルの構成要素（2/2）



- 属性 (Attribute)　→　**楕円**で表す．実体（または関連）の特性．例）学籍番号，氏名，科目名
  - 関連も属性を持てる．例）履修の「成績」（どの学生がどの科目で取ったかで決まる）
- 主キー　→　属性の**下線**で表す．第3回のリレーションの主キーと同じ（リレーション→実体型，タプル→実体と読み替える）

Topic 3

実体関連モデルの詳細

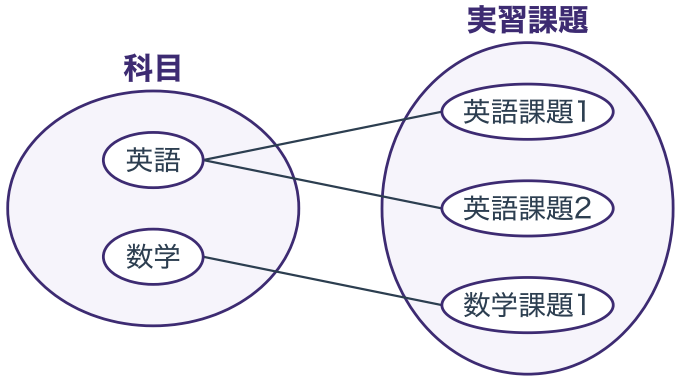
トピックのキーワード

- 多重度（1対多・多対多，最小値と最大値）
- 汎化階層（is-a）
- 表記法の違い（情報処理試験・UML）

いろいろな多重度：1対多



ER 図



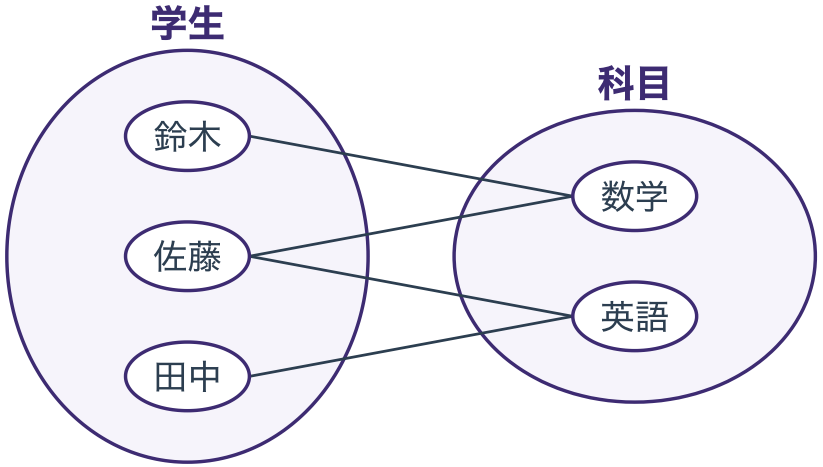
対応する現実世界のイメージ

- 一つの科目に複数の実習課題が対応する（実習課題は一つの科目だけに対応する）
- 関連における実体の個数の対応関係を**多重度**（濃度・基数制約）と呼ぶ．制約の一種．教科書では濃度（cardinality）
- 1とN（任意の自然数）を，関連型の線の**実体型の横**に書く．「科目1つに実習課題がN個」→Nを**実習課題の横**に

いろいろな多重度：多対多



ER 図



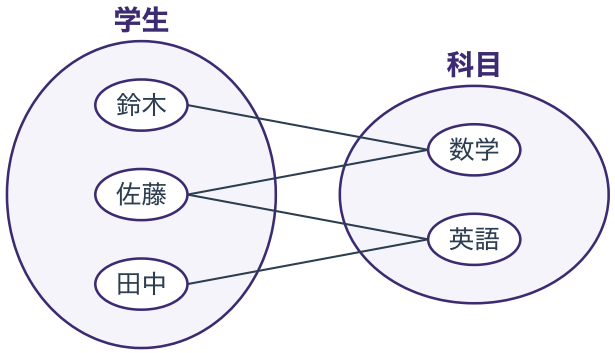
対応する現実世界のイメージ

- 一人の学生は複数の科目を履修できる → 科目の横に M
- 一つの科目を複数の学生が履修する → 学生の横に N
- **読み方**：「相手の実体1つに対して，こちら側の実体がいくつか」を，こちら側の横に書く

多重度の最小値と最大値



ER 図



対応する現実世界のイメージ

例えば，上記の例の場合，NやMの取りうる値の範囲が限られる場合がある．

- 学生は，最低10科目を履修しなければならない
- 学生は，最大24科目までしか履修できない
- 一つの科目を履修する最大人数は40人
- 5人未満しか希望者がいなかったら科目は実施されない

多重度の最小値と最大値



科目1つに学生 5〜40人

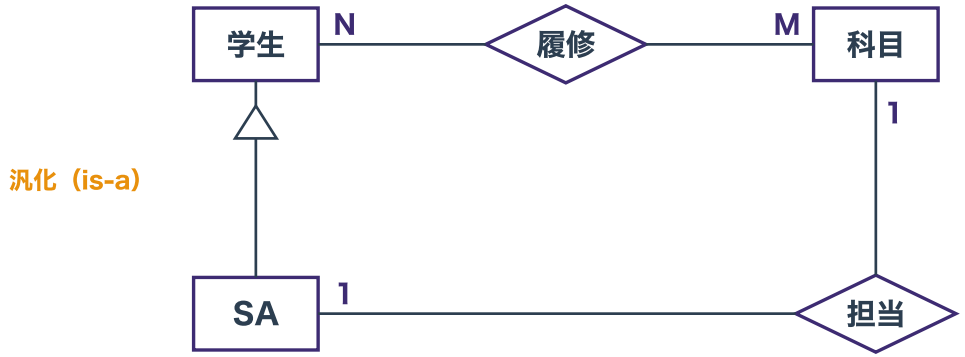
学生1人に科目 10〜24

前ページのような制約は，多重度の最小値と最大値で表現できる．本講義では **最小..最大** と書く（UML と同じ書き方．上限なしは *）

- 一つの科目を履修する人数は 5人以上，40人以下
  → 科目1つに対する学生の数なので，**学生の横**に 5..40
- 学生は 10科目以上，24科目以下を履修する
  → 学生1人に対する科目の数なので，**科目の横**に 10..24
- 最小が 0 なら「対応する相手がないこともある」．例）0..5：0〜5個．1..1：ちょうど1つ

汎化階層（is-a）

- SA（教員の手伝いをする学生）は，担当する科目を一つだけ持つ
  - 科目1つに SA は一人（SA の横の 1）
- 「SAは学生である(is a)」
  - 学生はSAの**汎化**（三角）
  - SAは学生の**特殊化**
- 汎化実体型の属性を特殊化実体型が継承する（SAは「学籍番号」を学生から継承）



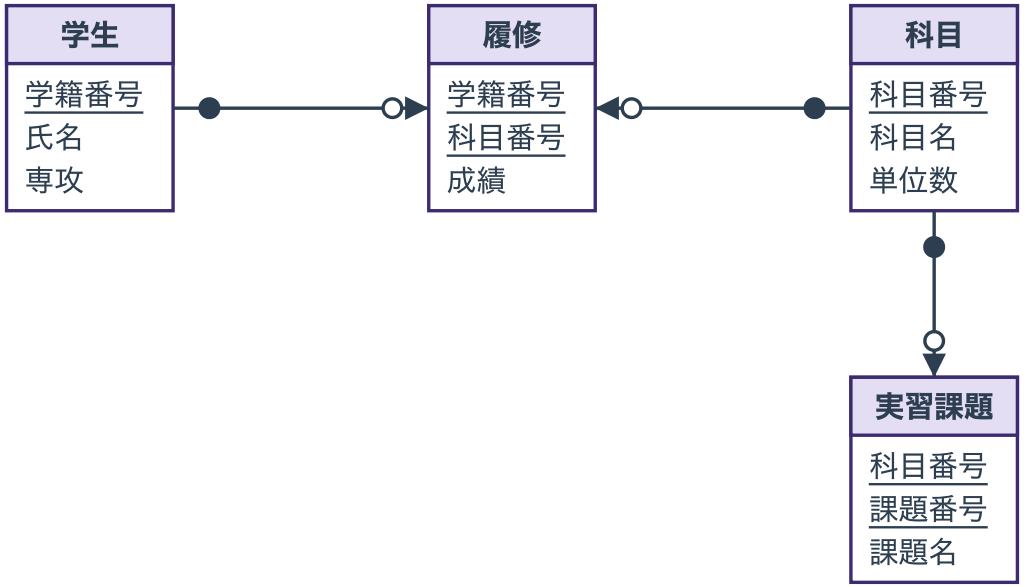
ER図の表記法に関する注意

(残念なことに)ER図には，様々な表記法があり統一されていない

- 今回採用しているのは，ERモデルを提案したP.Chenのオリジナルの表記法
  - 教科書での採用は多い
  - 属性を楕円で表すのは面倒（属性の数は大抵多いから）で，実用的にあまり採用されない表記法
- 重要な表記法として，さらに以下の2つがある
  - 情報処理試験に用いられる表記法
  - UMLにおけるクラス図

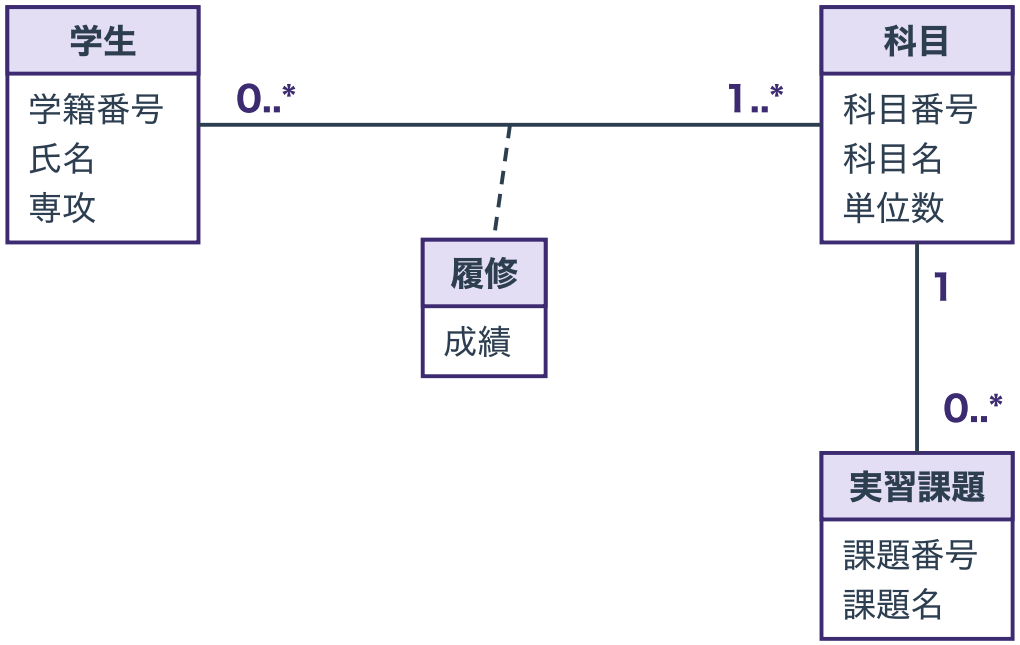
ER図の表記法：情報処理試験

- 実体型＝エンティティタイプ（四角），関連型＝リレーションシップ（線）
- 属性は四角の中に書く．キーは下線
- リレーションシップは属性を持てない → 履修をエンティティタイプにする
- 多重度は矢印（矢印の側が「多」）
- 最小値は白丸（0），黒丸（1以上）



ER図の表記法：UMLにおけるクラス図

- UML（Unified Modeling Language）：オブジェクト指向のソフトウェア設計の記法
- ER モデルはクラス図で書ける．実体型→クラス，関連型→関連
- 属性はクラスの四角の中．関連の属性（成績）は関連クラス（破線）
- 多重度は 最小値..最大値（* は上限なし）



ER図の表記法：どう考えるべきか

- 本講義・演習・試験では，教科書にならいChenの記法を使用することとする
- 将来的には，世の中での普及が進み，表現力も高いUMLを使うことが多くなると考えられる

多重度の表記法に関する注意

Chen の ER 図（本講義）



UML のクラス図



科目1つに学生 0人以上

学生1人に科目 1つ以上

- 本講義の Chen の図と UML は，多重度を書く**位置も書き方も同じ**
  - 相手1つに対するこちら側の個数を，こちら側に 最小..最大 で書く（1:N，情報処理試験の記法も同じ位置）

参考：参考書の (min, max) 記法は位置が逆

本講義：最小..最大（相手1つに対するこちら側の個数）



参考書：(min, max)（その実体が関連に何回参加するか）



学生は 1回以上 履修に参加

科目は 0回以上 履修に参加

- 北川（p.25）・Elmasri の **(min, max)** は「その実体が関連に何回参加するか」を表す．上と同じ制約でも**位置が逆**になる
- 「ER 図と UML は多重度の位置が逆」と言われるのはこの記法のこと．他の資料を読むときは，どちらの記法かを確かめる
- 本講義・試験は **最小..最大**（上の段）で書く

Topic 4

今回の演習と小テストの「型」

型I：表から ER 図を描く（例）

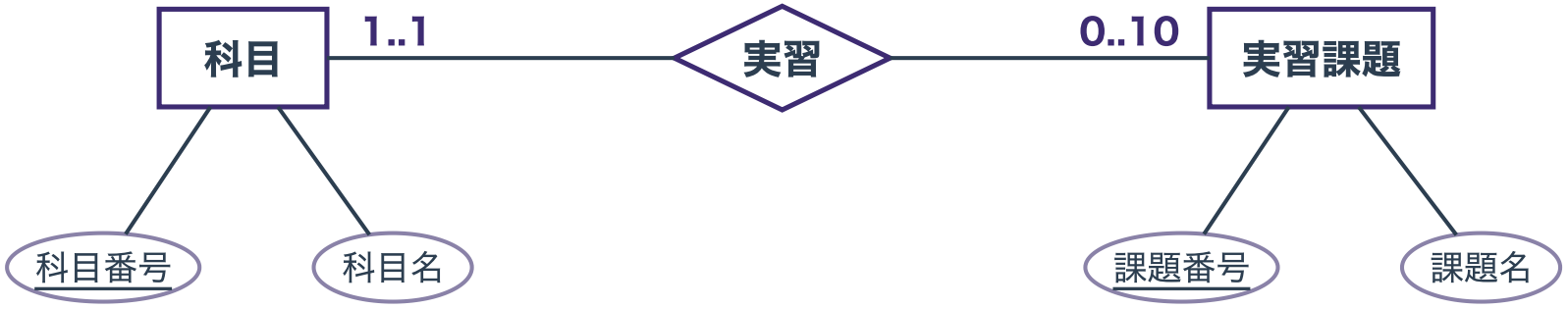
科目

科目番号	科目名
1	データベースの基礎
2	確率論

実習課題

課題番号	科目番号	課題名
1	1	SQL 演習
2	1	ER 図演習
3	2	確率の計算

外部キー → 関連型 実習 にする



条件：実習課題は必ず1つの科目に属する．科目には課題がないこともあり，最大10個

型I：表から ER 図を描く手順

1. テーブルごとに実体型（四角）を描く
2. **外部キーの列は属性にせず**，参照先との関連型（ひし形）にする
    - 例）実習課題の 科目番号 → 関連型 実習
3. 残りの列を属性（楕円）にし，主キーに下線を引く
4. 多重度：相手1つに対するこちら側の個数を，こちら側に 最小..最大 で書く
    - 課題1つに科目は1つ → **科目の横**に 1..1
    - 科目1つに課題は 0〜10個 → **実習課題の横**に 0..10

今回の小テストの「型」（小テスト2：11/25）

- **型I：ER 図を描く・読む**
  - 2つのテーブルから，実体型・関連型・属性・主キー（下線）を Chen の記法で描く
  - 条件文から，多重度を 最小..最大 で書く．**どちら側に書くか**に注意
- 手順は前のスライド（ポイント資料の表面にも同じ例がある）
- 第4・5回分は型F・G・H，第7回（正規化）分は次回

第6回演習と次回

- ポイント資料の裏面を，表面の例にならって自分の手で解く
  - 演習1：型Iの類題（学生・実習先企業の ER 図．関連型が2つ）
  - 確認問題：設計プロセスの順序，ER 図の読み取り
- 提出：表面・裏面を撮影して WebClass「第6回 演習」へ．締切：**次回（11/11）の講義開始30分後**
- 次回の予習：教科書 第7章 p.73〜（正規化理論：更新時異状と正規化）